

Desarrollo web y construcción de API

Breve descripción:

El componente formativo aborda los fundamentos del desarrollo web y la construcción de APIs, incluyendo conceptos de Internet, elementos físicos y lógicos de la red y tipos de conectividad. Asimismo, presenta bases de programación, arquitectura orientada a servicios, preparación del entorno de desarrollo, codificación de APIs, gestión de bases de datos y pruebas de software para garantizar el funcionamiento adecuado de las aplicaciones.

Tabla de contenido

Introducción	4
1. Fundamentos de Internet y del desarrollo web	7
1.1. Conceptos de Internet	8
1.2. Herramientas de Internet	10
1.3. Elementos físicos de Internet	12
1.4. Elementos lógicos de Internet	16
1.5. Tipos de conectividad en redes	19
2. Fundamentos de programación para el desarrollo web	27
2.1. Lenguajes de programación utilizados en el desarrollo web	27
2.2. Tipos de datos y estructuras de programación	34
2.3. Estructuras de selección y estructuras de repetición	36
2.4. Funciones, procedimientos, clases y objetos	38
3. Arquitectura orientada a servicios	41
3.1. Conceptos de arquitectura orientada a servicios	41
3.2. Servicios web SOAP	43
3.3. Servicios web REST	45
3.4. Aplicaciones y almacenamiento de datos en arquitecturas de servicios...	47

4.	Desarrollo de APIs y configuración del entorno.....	49
4.1.	Preparación del entorno de desarrollo.....	49
4.2.	Requerimientos de hardware y software de IDE.....	51
4.3.	Codificación de módulos y desarrollo de APIs.....	54
4.4.	Solicitudes, respuestas y autenticación en APIs REST.....	56
4.5.	Migraciones y sincronización de bases de datos	61
4.6.	ORM (Object Relational Mapping).....	63
5.	Pruebas en el desarrollo de APIs	67
5.1.	Conceptos de pruebas de software	68
5.2.	Características y tipos de pruebas	70
	Síntesis	73
	Glosario	75
	Referencias bibliográficas	77
	Créditos	79

Introducción

El desarrollo web ha evolucionado de manera significativa con el crecimiento de Internet y la necesidad de crear aplicaciones cada vez más dinámicas, interactivas y conectadas entre sí. En este contexto, el uso de lenguajes de programación y arquitecturas orientadas a servicios permite diseñar soluciones que facilitan el intercambio de información entre diferentes sistemas y plataformas.

Comprender los fundamentos de Internet, los elementos que lo conforman y los tipos de conectividad existentes resulta esencial para el desarrollo de aplicaciones web funcionales y eficientes. De igual forma, el manejo de conceptos básicos de programación, como los tipos de datos, las estructuras de control, las funciones y el uso de clases y objetos, permite estructurar adecuadamente la lógica de las aplicaciones.

Asimismo, la arquitectura orientada a servicios y el uso de servicios web como SOAP y REST facilitan la comunicación entre aplicaciones mediante APIs, permitiendo integrar servicios y gestionar el intercambio de datos de forma segura y eficiente. En este proceso también cobra importancia la preparación del entorno de desarrollo, la configuración de herramientas como los entornos de desarrollo integrados (IDE), la gestión de bases de datos y el uso de herramientas como los ORM.

Finalmente, las pruebas de software permiten verificar el correcto funcionamiento de las aplicaciones y asegurar la calidad de los servicios desarrollados, contribuyendo a la creación de soluciones tecnológicas confiables y eficientes en entornos digitales.

Para comprender la importancia del contenido y los temas abordados, se recomienda acceder al siguiente video:

Video 1. Desarrollo web y construcción de API



[Enlace de reproducción del video](#)

Transcripción del video. Desarrollo web y construcción de API

El desarrollo web ha transformado la manera en que las personas interactúan con la información, los servicios y las aplicaciones digitales. Desde los primeros sitios estáticos creados en la década de 1990 hasta las complejas plataformas que hoy conectan millones de usuarios en tiempo real, la evolución de Internet ha estado marcada por la necesidad de compartir datos de forma rápida, segura y estructurada.

En los inicios de la web, las páginas funcionaban principalmente como espacios informativos. Con el tiempo, el crecimiento de los servicios digitales, el comercio electrónico y las aplicaciones móviles impulsó la creación de nuevas formas de comunicación entre sistemas. En este contexto surgen las APIs o interfaces de programación de aplicaciones, que permiten que diferentes programas, plataformas y dispositivos intercambien información de manera eficiente.

Gracias a las APIs, hoy es posible integrar servicios de pago, sistemas de geolocalización, plataformas de mensajería o redes sociales dentro de una misma aplicación. Estas interfaces se han convertido en uno de los pilares de la arquitectura moderna del software, especialmente en entornos donde múltiples servicios deben comunicarse de forma constante.

Comprender cómo funcionan estas tecnologías permite interpretar la lógica detrás de los servicios digitales que se utilizan diariamente y abre la puerta al desarrollo de soluciones capaces de conectar sistemas, procesar información y crear aplicaciones cada vez más dinámicas e interconectadas.

1. Fundamentos de Internet y del desarrollo web

El desarrollo web y la construcción de Interfaces de Programación de Aplicaciones (API), se fundamentan en el funcionamiento de Internet como infraestructura global de comunicación. Comprender cómo se interconectan los dispositivos, cómo se transmiten los datos y qué tecnologías permiten acceder a los recursos digitales resulta esencial para diseñar aplicaciones y servicios que operen en entornos distribuidos.

Internet permite que millones de dispositivos alrededor del mundo intercambien información de manera rápida y eficiente mediante protocolos de comunicación estandarizados. Sobre esta infraestructura se desarrollan aplicaciones web, plataformas digitales, servicios en la nube y APIs que facilitan la integración entre sistemas.

El desarrollo de Internet es el resultado de múltiples investigaciones realizadas desde la década de 1960, orientadas a mejorar la comunicación entre computadores.

Entre los principales aportes históricos se destacan:

- **Leonard Kleinrock y Paul Baran**

A comienzos de los años sesenta, Kleinrock propuso dividir la información en pequeños paquetes de datos que pudieran transmitirse de manera independiente a través de una red. Este enfoque permitió mejorar la eficiencia en la transmisión de información. Por su parte, Paul Baran planteó el concepto de redes descentralizadas, en las cuales los datos pueden viajar por diferentes rutas hasta llegar a su destino.

- **Lawrence Roberts**

Basado en estos avances, Roberts diseñó en 1966 la primera red de conmutación de paquetes, que posteriormente se convertiría en ARPANET, considerada la red precursora de Internet. En 1969 se logró establecer la primera conexión entre dos computadoras ubicadas en distintas instituciones.

- **Vinton Cerf y Bob Kahn**

En 1974 desarrollaron el protocolo TCP/IP, que permitió la comunicación entre diferentes redes y sistemas informáticos. Este protocolo se convirtió en el estándar que hizo posible la expansión global de Internet.

- **Tim Berners-Lee**

En 1989 propuso la creación de la World Wide Web (WWW) y desarrolló tecnologías fundamentales como HTML y HTTP, que permiten estructurar y transferir información a través de navegadores web.

Gracias a estos avances, Internet evolucionó hasta convertirse en la infraestructura tecnológica que hoy soporta aplicaciones web, servicios digitales, plataformas en la nube y el desarrollo de interfaces de programación de aplicaciones.

1.1. Conceptos de Internet

Internet es una red global de redes que permite la interconexión de millones de dispositivos alrededor del mundo para el intercambio de información y la prestación de diversos servicios digitales. Su funcionamiento se basa en infraestructuras tecnológicas,

protocolos de comunicación y servicios que facilitan la transmisión de datos entre sistemas conectados.

Esta red utiliza principalmente el conjunto de protocolos TCP/IP, que establece las reglas para la transmisión de datos entre dispositivos. Gracias a estos protocolos, la información se divide en paquetes que pueden viajar por diferentes rutas dentro de la red hasta llegar a su destino final, donde son reorganizados para reconstruir el mensaje original.

En el contexto del desarrollo web, Internet constituye la infraestructura sobre la cual operan las aplicaciones, los servicios digitales y las Interfaces de Programación de Aplicaciones (API).

Entre los conceptos fundamentales asociados al funcionamiento de Internet se encuentran:

- **Red:** conjunto de dispositivos informáticos interconectados, como computadores, servidores, routers y dispositivos móviles, que se comunican entre sí para compartir información, recursos y servicios mediante diferentes tecnologías de conexión.
- **Protocolo:** conjunto de reglas y estándares que definen cómo se transmiten, reciben y procesan los datos entre los dispositivos conectados a una red. Los protocolos garantizan que los sistemas puedan comunicarse de manera organizada y compatible.
- **Cliente:** dispositivo, programa o aplicación que solicita servicios o recursos a un servidor dentro de una red. Por ejemplo, un navegador web actúa como cliente cuando solicita una página a un servidor web.

- **Dirección IP:** identificador numérico único asignado a cada dispositivo conectado a una red que utiliza el protocolo IP. Esta dirección permite localizar y reconocer los equipos dentro de la red para que puedan enviar y recibir información correctamente.
- **Servidor:** equipo o sistema informático que proporciona servicios, recursos o información a otros dispositivos dentro de una red. Los servidores pueden ofrecer distintos servicios, como alojamiento de páginas web, almacenamiento de archivos o gestión de correos electrónicos.
- **Paquete de datos:** unidad básica de información que se transmite a través de una red. Los datos se dividen en pequeños paquetes que viajan por diferentes rutas hasta llegar a su destino, donde se reorganizan para reconstruir el mensaje original.

En conjunto, estos conceptos permiten comprender el funcionamiento de Internet como infraestructura tecnológica que soporta la comunicación digital. Su conocimiento resulta fundamental para el desarrollo web, ya que las aplicaciones, los servicios en línea y las Interfaces de Programación de Aplicaciones (API) operan sobre esta red global y permiten el intercambio de información entre diferentes sistemas y plataformas.

1.2. Herramientas de Internet

El funcionamiento de Internet y de las aplicaciones web depende de diversas herramientas tecnológicas que permiten acceder a la información, procesar solicitudes y gestionar la comunicación entre los diferentes sistemas conectados a la red. Estas

herramientas intervienen en el intercambio de datos entre los usuarios y los servidores, facilitando la navegación, la publicación de contenidos y la integración de servicios digitales.

Entre las principales herramientas utilizadas en el entorno de Internet se encuentran:

- **Navegador web:** aplicación o software instalado en un dispositivo que permite acceder, visualizar e interactuar con la información disponible en Internet. Los navegadores interpretan los lenguajes utilizados en la web, como HTML, CSS y JavaScript, transformándolos en páginas visuales con texto, imágenes, videos y otros elementos interactivos.
- **Servidor web:** sistema informático encargado de almacenar, procesar y entregar los recursos de un sitio web a los usuarios que los solicitan. Cuando un navegador realiza una petición, el servidor recibe la solicitud, localiza la información requerida y la envía al cliente para su visualización mediante protocolos de comunicación web.
- **Cliente HTTP:** aplicación o componente de software que envía solicitudes a un servidor web utilizando el protocolo HTTP o HTTPS con el fin de acceder a recursos como páginas web, archivos o servicios de datos. Los navegadores web y algunas herramientas de desarrollo funcionan como clientes HTTP al iniciar la comunicación con los servidores.
- **DNS (Sistema de Nombres de Dominio):** sistema que traduce los nombres de dominio utilizados por las personas en direcciones IP numéricas que identifican a los servidores en Internet. Gracias al DNS, los usuarios pueden

acceder a los servicios web mediante nombres fáciles de recordar, en lugar de utilizar direcciones numéricas complejas.

Estas herramientas permiten el funcionamiento del modelo cliente-servidor característico de la web. En este modelo, el usuario accede a los servicios mediante un navegador o cliente HTTP, el servidor web procesa la solicitud y el sistema DNS facilita la localización de los recursos dentro de la infraestructura de Internet.

1.3. Elementos físicos de Internet

La infraestructura de Internet se basa en una serie de componentes físicos que permiten transportar la información entre dispositivos y redes distribuidas en todo el mundo. Estos elementos corresponden a equipos y dispositivos tangibles encargados de transmitir señales, dirigir el tráfico de datos y proporcionar los servicios necesarios para el funcionamiento de aplicaciones, sitios web y plataformas digitales.

Los elementos físicos constituyen la base material de Internet, ya que permiten que los datos viajen a través de diferentes medios de comunicación, como cables, enlaces inalámbricos o conexiones satelitales. Entre los principales dispositivos que hacen parte de esta infraestructura se encuentran routers, antenas y servidores, los cuales cumplen funciones específicas dentro de la red.

a) Router

Un router o enrutador es un dispositivo de red encargado de conectar diferentes redes informáticas y dirigir el tráfico de datos entre ellas. Su función principal consiste en determinar la mejor ruta para que los paquetes de información lleguen desde el dispositivo de origen hasta su destino dentro de Internet o de una red local.

En entornos domésticos y empresariales, el router actúa como el punto central de conexión, distribuyendo la señal de Internet mediante conexiones cableadas o inalámbricas (Wi-Fi) hacia diversos dispositivos como computadores, teléfonos móviles, televisores inteligentes y otros equipos conectados.

Entre sus principales funciones se encuentran el enrutamiento de paquetes de datos mediante tablas de rutas, la asignación de direcciones IP a los dispositivos conectados a la red mediante protocolos como DHCP y la implementación de mecanismos básicos de seguridad, como la traducción de direcciones de red (NAT) o firewalls integrados.

Existen diferentes tipos de routers según su uso y capacidad. Los core routers son utilizados por proveedores de servicios de Internet en el núcleo de la red para gestionar grandes volúmenes de tráfico. Los edge routers se ubican en los límites de las redes organizacionales para conectar con otras redes externas. Por su parte, los routers SOHO (Small Office/Home Office) son los dispositivos comunes en hogares y pequeñas oficinas, donde integran funciones de router, switch y punto de acceso inalámbrico.

b) Antenas

Las antenas son dispositivos diseñados para emitir o recibir ondas electromagnéticas, permitiendo la transmisión de información a través del aire mediante señales inalámbricas. Su función consiste en convertir la energía eléctrica en ondas de radio y, a su vez, transformar las ondas recibidas nuevamente en señales eléctricas que pueden ser interpretadas por los dispositivos de comunicación.

En el contexto de Internet, las antenas son fundamentales para las redes inalámbricas, ya que permiten distribuir la señal en entornos donde no es posible utilizar cableado físico o donde se requiere cobertura en amplias áreas geográficas. Estas se emplean en redes Wi-Fi, enlaces de radio, redes móviles y comunicaciones satelitales.

Las antenas pueden clasificarse según su forma de propagación de la señal. Las antenas omnidireccionales transmiten la señal en todas las direcciones y suelen utilizarse en redes domésticas o en espacios interiores. Las antenas direccionales, como las antenas parabólicas o Yagi, concentran la señal en una dirección específica y se emplean para enlaces de mayor distancia. También existen antenas sectoriales, utilizadas comúnmente en torres de telecomunicaciones para cubrir áreas determinadas dentro de las redes móviles.

Entre sus características técnicas más relevantes se encuentran la ganancia, que indica el alcance o intensidad de la señal; la frecuencia de operación, como 2.4 GHz, 5 GHz o 6 GHz; y la polarización, que corresponde a la orientación de las ondas electromagnéticas transmitidas.

c) Servidores

Los servidores son equipos informáticos de alto rendimiento diseñados para proporcionar recursos, datos o servicios a otros dispositivos dentro de una red. Estos equipos almacenan información y ejecutan aplicaciones que pueden ser solicitadas por los usuarios a través de Internet.

Cuando un usuario accede a una página web, envía una solicitud desde su dispositivo a un servidor que almacena el contenido del sitio. El servidor procesa la

solicitud y envía los datos necesarios para que el navegador del usuario pueda visualizar la información solicitada.

Los servidores pueden desempeñar diferentes funciones dependiendo del servicio que ofrecen. Los servidores web almacenan y entregan páginas web y recursos asociados como imágenes o archivos. Los servidores de bases de datos gestionan información estructurada utilizada por aplicaciones y sistemas informáticos. Los servidores de correo administran el envío y recepción de mensajes electrónicos mediante protocolos especializados. Asimismo, los servidores DNS permiten traducir nombres de dominio en direcciones IP para facilitar el acceso a los recursos en Internet.

Estos equipos suelen contar con hardware especializado que garantiza un funcionamiento continuo y estable. Entre sus características más importantes se encuentran procesadores de alto rendimiento, memoria con corrección de errores (ECC), sistemas de almacenamiento redundante y fuentes de alimentación duplicadas. Estas características permiten mantener los servicios disponibles de forma permanente, asegurando que las aplicaciones, sitios web y plataformas digitales puedan operar las veinticuatro horas del día.

En la actualidad, muchos de estos servidores se encuentran alojados en centros de datos o en infraestructuras de computación en la nube, lo que permite escalar los servicios y distribuir las cargas de trabajo entre múltiples sistemas. Sin embargo, aunque la infraestructura pueda estar virtualizada, los principios de funcionamiento de estos elementos físicos continúan siendo fundamentales para el funcionamiento de Internet y para la prestación de servicios digitales.

1.4. Elementos lógicos de Internet

Además de los componentes físicos que permiten la transmisión de datos, Internet también se basa en una serie de elementos lógicos que organizan, identifican y regulan la comunicación entre los dispositivos conectados a la red. Estos elementos corresponden a configuraciones, direcciones y protocolos que establecen las reglas necesarias para que los sistemas puedan intercambiar información de manera estructurada y eficiente.

Los elementos lógicos permiten identificar los dispositivos dentro de la red, establecer canales de comunicación entre aplicaciones y definir los estándares mediante los cuales se transmiten los datos. Entre los principales elementos lógicos utilizados en Internet se encuentran las direcciones IP, los puertos de comunicación y los protocolos de red.

a) Dirección IP

La dirección IP (Internet Protocol) es un identificador numérico único asignado a cada dispositivo conectado a una red que utiliza el protocolo IP. Su función principal es permitir la identificación y localización de los equipos dentro de la red, facilitando el envío y la recepción de información entre diferentes sistemas.

Cada dispositivo conectado a Internet, como computadores, servidores, teléfonos móviles o dispositivos inteligentes, requiere una dirección IP para poder comunicarse con otros equipos. Esta dirección funciona de manera similar a una dirección postal, ya que permite que los paquetes de datos lleguen al destino correcto dentro de la red.

Las direcciones IP pueden presentarse en diferentes formatos. El más común es IPv4, que utiliza una combinación de cuatro números separados por puntos.

Ejemplo de dirección IPv4: 192.168.1.10

En la actualidad también se utiliza IPv6, un formato más reciente que permite una mayor cantidad de direcciones disponibles para soportar el crecimiento de los dispositivos conectados a Internet.

b) Puertos

Los puertos son canales de comunicación que permiten que diferentes aplicaciones o servicios puedan intercambiar información dentro de un mismo dispositivo conectado a la red. Cada puerto se identifica mediante un número y actúa como un punto específico de entrada o salida para los datos.

Cuando un dispositivo recibe información desde Internet, el número de puerto indica a qué aplicación o servicio debe entregarse esa información. De esta manera, un mismo equipo puede ejecutar simultáneamente varios servicios de red sin interferencias entre ellos.

Algunos puertos son utilizados de manera estándar por servicios ampliamente conocidos en Internet.

Tabla 1. Puertos de uso frecuente en entornos web

Puerto	Uso habitual
80	HTTP
443	HTTPS
3306	MySQL

El uso de puertos permite organizar el tráfico de datos dentro de los dispositivos y garantizar que cada servicio pueda funcionar correctamente dentro de la infraestructura de red.

c) Protocolos de red

Los protocolos de red son conjuntos de reglas y estándares que definen cómo se transmiten, reciben y procesan los datos entre los dispositivos conectados a Internet. Estos protocolos garantizan que sistemas con diferentes tipos de hardware, software y fabricantes puedan comunicarse entre sí de forma compatible.

La comunicación en Internet se basa principalmente en la familia de protocolos TCP/IP, que constituye el fundamento de la transmisión de datos en la red. El protocolo TCP (Transmission Control Protocol) se encarga de dividir la información en paquetes, controlar su envío y verificar que los datos lleguen correctamente al destino. Por su parte, el protocolo IP (Internet Protocol) se encarga de dirigir los paquetes hacia la dirección correcta dentro de la red.

Además de TCP/IP, existen otros protocolos que permiten el funcionamiento de diversos servicios y aplicaciones en Internet.

Tabla 2. Protocolos fundamentales para servicios y aplicaciones

Protocolo	Función principal.
HTTP	Transferencia de recursos web.
HTTPS	Transferencia segura de recursos web.
TCP/IP	Base de la comunicación y el direccionamiento en Internet.

Protocolo	Función principal.
FTP	Transferencia de archivos.

En conjunto, estos elementos lógicos permiten que los dispositivos conectados a Internet puedan identificarse, establecer canales de comunicación y transmitir información de manera organizada. Gracias a estas configuraciones y estándares, es posible que aplicaciones, sitios web y servicios digitales funcionen correctamente dentro de la infraestructura global de Internet.

1.5. Tipos de conectividad en redes

La conectividad en redes se refiere a la capacidad que tienen los dispositivos para establecer comunicación entre sí y acceder a los recursos disponibles en Internet. Esta conectividad depende del medio de transmisión utilizado, la infraestructura disponible y las tecnologías empleadas para transportar la información.

La calidad de la conexión puede variar según factores como el ancho de banda, la latencia, la estabilidad del enlace y la distancia entre los dispositivos conectados. Estos aspectos influyen directamente en el rendimiento de aplicaciones, plataformas digitales y servicios en línea.

Actualmente existen diferentes tipos de conectividad que permiten a los usuarios y a las organizaciones acceder a Internet desde diversos entornos.

a) Conectividad por cable o banda ancha

La banda ancha por cable es una de las formas más utilizadas para acceder a Internet en hogares y empresas. Utiliza infraestructuras de cable coaxial o fibra óptica para transmitir grandes volúmenes de datos a alta velocidad.

Figura 1. Cable coaxial



Nota. Tomado de Xataka Basics (2026).

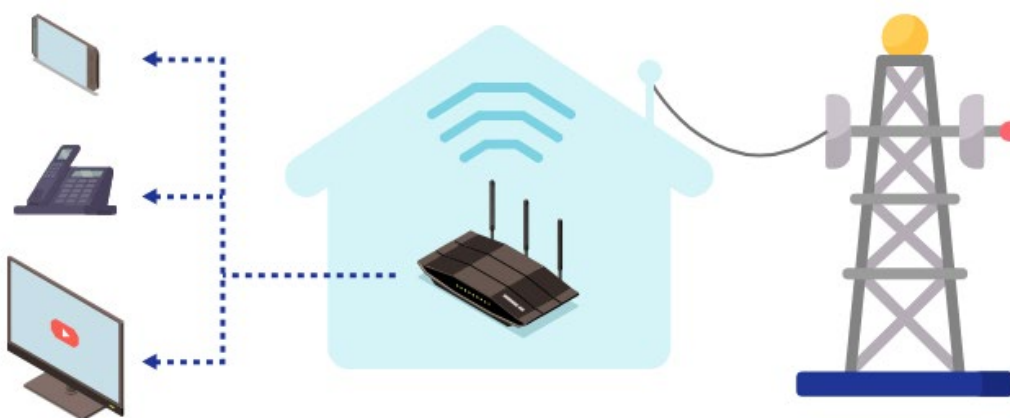
Este tipo de conexión es ofrecido por los proveedores de servicios de Internet (ISP) y permite una comunicación estable y continua entre los usuarios y la red global. Gracias a su capacidad de transmisión, es adecuada para actividades que requieren alto consumo de datos, como videoconferencias, plataformas en la nube o transmisión de contenidos multimedia.

b) Conexiones DSL

Las conexiones DSL (Digital Subscriber Line) utilizan las líneas telefónicas tradicionales para transmitir datos digitales. Aunque su velocidad es menor en

comparación con las conexiones de fibra óptica o cable coaxial, continúa siendo una alternativa utilizada en zonas donde otras infraestructuras de red no están disponibles.

Figura 2. Cómo funciona el servicio de internet DSL



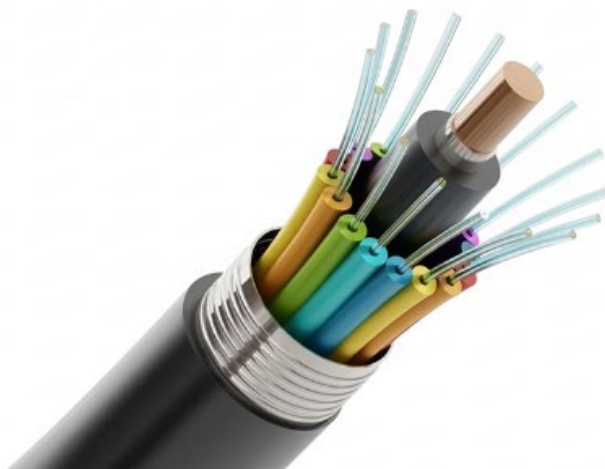
Nota. Tomado de Webformix (2021).

Esta tecnología permite que la transmisión de datos y la comunicación telefónica se realicen simultáneamente sobre la misma línea, sin interferencias entre ambos servicios.

c) Internet por fibra óptica

La fibra óptica es una de las tecnologías de conectividad más avanzadas en la actualidad. Utiliza cables de vidrio o plástico que transmiten información mediante impulsos de luz, lo que permite alcanzar velocidades de transmisión muy elevadas y una menor latencia en comparación con otros medios de transmisión.

Figura 3. Fibra óptica



Nota. Tomado de The Network Installers (2024).

Debido a estas características, la fibra óptica se emplea ampliamente en redes empresariales, centros de datos y servicios de computación en la nube, donde se requiere una transferencia rápida y confiable de grandes volúmenes de información.

d) Redes móviles

Permiten el acceso a Internet mediante tecnologías de comunicación celular. Estas redes utilizan infraestructuras de antenas y estaciones base que transmiten señales inalámbricas hacia dispositivos como teléfonos inteligentes, tabletas y módems portátiles.

Figura 4. Antenas de telefonía móvil



Nota. Tomado de Ubigo (2024).

Las tecnologías móviles han evolucionado significativamente con el tiempo, pasando por diferentes generaciones como 3G, 4G y 5G, cada una con mejoras en velocidad de transmisión, capacidad de conexión y eficiencia en el uso del espectro radioeléctrico.

Gracias a estas tecnologías, los usuarios pueden acceder a Internet desde prácticamente cualquier lugar con cobertura de red móvil.

e) Internet por satélite

El acceso a Internet por satélite se utiliza principalmente en regiones donde las infraestructuras terrestres de telecomunicaciones son limitadas o inexistentes. En este sistema, los datos se transmiten entre el dispositivo del usuario y un satélite ubicado en órbita terrestre, el cual se comunica con estaciones terrestres conectadas a Internet.

Figura 5. Servicio satelital



Nota. Tomado de BusinessCom (s. f.).

Aunque este tipo de conexión puede presentar mayor latencia debido a la distancia que deben recorrer las señales, constituye una solución importante para garantizar conectividad en zonas rurales o de difícil acceso.

f) Redes inalámbricas Wi-Fi

Es una tecnología que permite conectar dispositivos a Internet de forma inalámbrica dentro de un área determinada, como hogares, oficinas, universidades o espacios públicos. Esta tecnología utiliza ondas de radio para transmitir la información entre el router o punto de acceso y los dispositivos conectados.

Figura 6. Una red de área local con varios dispositivos conectados



Nota. Tomado de Harrison Gough (2024), Avast.

Las redes Wi-Fi facilitan la conexión simultánea de múltiples dispositivos sin necesidad de cableado físico, lo que las convierte en una solución ampliamente utilizada en entornos domésticos y empresariales.

Los diferentes tipos de conectividad presentan características particulares relacionadas con la velocidad de transmisión de datos, el alcance geográfico y el medio físico o tecnológico utilizado para la comunicación. Estas diferencias influyen directamente en el desempeño de los servicios digitales, el acceso a aplicaciones en línea y la estabilidad de la conexión. A continuación, se presenta una comparación general de algunos tipos de conectividad utilizados en redes, con el fin de identificar sus principales características y aplicaciones.

Tabla 3. Comparación general de algunos tipos de conectividad

Tipo de conectividad	Velocidad aproximada	Alcance	Medio de transmisión
DSL	Media	Medio	Línea telefónica

Tipo de conectividad	Velocidad aproximada	Alcance	Medio de transmisión
Wi-Fi	Media	Bajo	Inalámbrico
Banda ancha por cable	Alta	Medio	Cable coaxial
Fibra óptica	Muy alta	Alto	Fibra óptica
Satélite	Media	Muy alto	Señal satelital
Redes móviles	Media a alta	Alto	Inalámbrico celular

En el diseño de aplicaciones y servicios digitales es importante considerar el tipo de conectividad disponible para los usuarios, ya que factores como la velocidad de la red, la latencia y la estabilidad de la conexión pueden influir en la experiencia de uso, el rendimiento de las aplicaciones y la disponibilidad de los servicios en línea.

2. Fundamentos de programación para el desarrollo web

Los fundamentos de programación para el desarrollo web permiten comprender cómo se diseñan y construyen aplicaciones que funcionan en entornos digitales y en Internet. A través de los lenguajes de programación, los desarrolladores pueden crear instrucciones que indican a los sistemas informáticos cómo procesar datos, responder a las solicitudes de los usuarios y generar contenidos dinámicos dentro de los sitios y aplicaciones web.

En el desarrollo web, la programación cumple un papel fundamental, ya que permite implementar la lógica que controla el comportamiento de las aplicaciones, gestionar la información que circula entre el cliente y el servidor, y automatizar procesos dentro de los sistemas digitales. Para lograrlo, se utilizan diferentes lenguajes y herramientas que facilitan la construcción de aplicaciones escalables, interactivas y conectadas a servicios en línea.

En este contexto, resulta necesario comprender algunos conceptos esenciales de la programación, como los lenguajes utilizados en el desarrollo web, los tipos de datos y las estructuras de programación, las estructuras de control que permiten tomar decisiones o repetir procesos, y los principios de la programación orientada a objetos. Estos fundamentos constituyen la base para el diseño y desarrollo de aplicaciones web modernas y servicios que operan en entornos distribuidos y en la nube.

2.1. Lenguajes de programación utilizados en el desarrollo web

El desarrollo web combina diferentes tecnologías que permiten estructurar la información, diseñar la interfaz de usuario, implementar la lógica de interacción y

procesar datos en el servidor. Estas tecnologías trabajan de forma integrada para construir aplicaciones web dinámicas capaces de responder a las solicitudes de los usuarios y gestionar información en tiempo real.

Un lenguaje de programación es un sistema de instrucciones que permite a los desarrolladores comunicar al computador las operaciones que debe ejecutar. A través de estos lenguajes es posible crear software, automatizar procesos, procesar datos y desarrollar aplicaciones que operan en entornos digitales y en Internet.

En el contexto del desarrollo web, los lenguajes se utilizan para distintas funciones, como estructurar el contenido de una página, definir su apariencia visual, implementar interactividad o procesar información en servidores que gestionan bases de datos y servicios en línea.

Entre las principales razones para utilizar lenguajes de programación se encuentran:

- Permitir la creación de aplicaciones y sistemas informáticos.
- Automatizar tareas y procesos repetitivos.
- Procesar y analizar grandes volúmenes de datos.
- Controlar dispositivos y servicios tecnológicos.
- Desarrollar soluciones innovadoras en entornos digitales.

En el desarrollo web moderno, diferentes lenguajes cumplen roles específicos dentro de la arquitectura de las aplicaciones.

a) HTML y CSS

HTML (HyperText Markup Language) es el lenguaje estándar utilizado para estructurar el contenido de las páginas web. Los navegadores interpretan el código HTML y lo presentan visualmente al usuario.

Aunque HTML no se considera un lenguaje de programación en sentido estricto, es fundamental para el desarrollo web, ya que define la estructura semántica de los elementos que conforman una página, como encabezados, párrafos, enlaces, imágenes y tablas.

Algunos ejemplos de etiquetas HTML son:

- `<h1>` a `<h6>` para encabezados.
- `<p>` para párrafos.
- `<a>` para enlaces.
- `<img>` para imágenes.
- `<table>` para tablas.

Para complementar la estructura definida por HTML se utiliza CSS (Cascading Style Sheets), un lenguaje de estilo que permite definir la presentación visual de los elementos.

CSS permite controlar aspectos como:

- Colores y fondos.
- Tipografías y tamaños de letra.
- Márgenes y espaciados.
- Diseño de la página mediante modelos como flexbox o grid.

La combinación de HTML para la estructura y CSS para la presentación constituye la base de la interfaz visual de cualquier sitio web.

b) JavaScript

Es uno de los lenguajes de programación más importantes del desarrollo web moderno. Su función principal es añadir interactividad y dinamismo a las páginas web, permitiendo que respondan a las acciones del usuario.

Con JavaScript es posible implementar funciones como:

- Validación de formularios.
- Actualización dinámica de contenido.
- Animaciones e interacciones.
- Comunicación con servidores mediante APIs.

El ecosistema de JavaScript incluye diversas bibliotecas y frameworks que facilitan el desarrollo de aplicaciones web modernas. Entre los más conocidos se encuentran:

- **jQuery**: biblioteca que simplifica la manipulación del DOM.
- **React**: biblioteca basada en componentes para construir interfaces interactivas.
- **Angular**: framework desarrollado por Google para crear aplicaciones web complejas.

Gracias a su versatilidad, JavaScript puede ejecutarse tanto en el navegador como en el servidor mediante tecnologías como Node.js.

c) Python

Es un lenguaje de programación de propósito general ampliamente utilizado en desarrollo web, ciencia de datos, automatización y desarrollo de software.

Se caracteriza por su sintaxis clara y legible, lo que facilita la comprensión del código y el trabajo colaborativo entre desarrolladores.

Entre los frameworks más utilizados para desarrollo web se encuentran:

- **Django:** framework completo que facilita el desarrollo rápido de aplicaciones web.
- **Flask:** microframework ligero y flexible.
- **Pyramid:** framework adaptable a proyectos de distintos tamaños.

Python permite desarrollar aplicaciones web escalables y eficientes, por lo que es ampliamente utilizado en plataformas digitales y servicios en la nube.

d) PHP (Hypertext Preprocessor)

Es un lenguaje de programación ampliamente utilizado para el desarrollo de aplicaciones web del lado del servidor.

Este lenguaje permite generar contenido dinámico en páginas web, gestionar sesiones de usuario, procesar formularios e interactuar con bases de datos.

Entre los frameworks más utilizados en PHP se encuentran:

- **Laravel:** conocido por su elegancia y facilidad de uso.
- **Symfony:** framework modular orientado a proyectos complejos.
- **CodeIgniter:** framework ligero de alto rendimiento.

PHP es reconocido por su facilidad de aprendizaje, su amplia comunidad de desarrolladores y su compatibilidad con numerosos servidores web.

e) Ruby

Es un lenguaje de programación orientado a objetos que destaca por su simplicidad y productividad en el desarrollo de software.

Su principal herramienta para desarrollo web es Ruby on Rails, un framework que facilita la creación rápida de aplicaciones mediante el patrón de arquitectura Modelo-Vista-Controlador (MVC).

Ruby se caracteriza por:

- Sintaxis clara y elegante.
- Desarrollo rápido de aplicaciones.
- Fuerte comunidad de desarrolladores.

f) Java

Es un lenguaje de programación robusto y altamente escalable utilizado en aplicaciones empresariales y sistemas de gran tamaño.

En el desarrollo web se emplea principalmente para construir aplicaciones backend que requieren alta estabilidad y capacidad de procesamiento.

Entre sus frameworks más utilizados se encuentran:

- **Spring Framework:** ampliamente utilizado en aplicaciones empresariales.
- **JavaServer Faces (JSF):** framework para el desarrollo de interfaces web basadas en componentes.

Java destaca por su portabilidad, seguridad y capacidad para manejar aplicaciones de gran escala.

La diversidad de lenguajes y tecnologías utilizadas en el desarrollo web responde a los diferentes roles que cada uno cumple dentro de una aplicación. Mientras algunos se orientan a la estructura y presentación de la interfaz, otros se enfocan en la lógica de interacción o en el procesamiento de datos en el servidor. A continuación, se presenta un panorama general de algunas tecnologías ampliamente utilizadas en el desarrollo web y sus principales funciones.

Tabla 4. Panorama general de tecnologías frecuentes en desarrollo web

Tecnología	Rol principal	Ejemplos o frameworks asociados
HTML/CSS	Estructura y presentación de la interfaz.	HTML5, CSS3, Flexbox y Grid
JavaScript	Interactividad en cliente y servidor.	React, Angular y Node.js
Python	Backend y automatización.	Django, Flask y Pyramid
PHP	Backend web y generación dinámica.	Laravel, Symfony y CodeIgniter
Ruby	Desarrollo web orientado a productividad.	Ruby on Rails y Sinatra
Java	Aplicaciones empresariales y backend robusto.	Spring y JSF

En el desarrollo web moderno, estos lenguajes se integran con infraestructuras de computación en la nube, contenedores, servicios administrados y plataformas de

despliegue automatizado. Por esta razón, el lenguaje de programación representa solo una parte del ecosistema tecnológico, mientras que la arquitectura del sistema, la seguridad y las estrategias de despliegue también influyen en el funcionamiento y la escalabilidad de las aplicaciones.

2.2. Tipos de datos y estructuras de programación

Toda aplicación informática necesita representar información y operar sobre ella. Para lograrlo, los lenguajes de programación utilizan tipos de datos, que definen la naturaleza de los valores que se almacenan, y estructuras de programación, que permiten organizar y manipular conjuntos de datos de manera eficiente.

Esta distinción es fundamental en el desarrollo de aplicaciones web y servicios digitales, ya que facilita el modelado de entidades, la organización de la información y la transmisión de datos entre sistemas mediante formatos como JSON o XML.

Los tipos de datos representan la forma en que se almacena y manipula la información dentro de un programa. Cada tipo establece qué clase de valores puede contener una variable y qué operaciones se pueden realizar sobre dichos valores.

Tabla 5. Tipos de datos de uso frecuente

Tipo de dato	Descripción	Ejemplo
Numérico	Representa números enteros o decimales utilizados en operaciones matemáticas.	10, 25.5
Cadena de texto	Representa caracteres, palabras o frases.	"Hola mundo"
Booleano	Representa valores lógicos utilizados en decisiones del programa.	Verdadero / Falso

Tipo de dato	Descripción	Ejemplo
Fecha/Hora	Representa información temporal como fechas o momentos específicos.	2026-04-08

El uso adecuado de los tipos de datos permite organizar correctamente la información, evitar errores durante el procesamiento y realizar operaciones coherentes con la naturaleza de cada valor.

Las estructuras de programación son mecanismos que permiten organizar y gestionar múltiples datos dentro de un sistema. A través de estas estructuras es posible trabajar con conjuntos de información de forma más eficiente y representar entidades complejas dentro de una aplicación.

En programación se pueden identificar diferentes tipos de estructuras según la forma en que almacenan la información.

Tabla 6. Tipos de estructuras de datos

Tipo de estructura	Característica	Ejemplo conceptual
Estructuras simples.	Almacenan un solo valor.	Un número o un texto
Listas o arreglos.	Colección ordenada de elementos del mismo tipo.	[1, 2, 3, 4]
Registros o estructuras clave-valor.	Permiten asociar valores a identificadores.	Nombre → "Juan"

Estas estructuras permiten organizar grandes volúmenes de información, facilitar la búsqueda y manipulación de datos y representar elementos del mundo real como usuarios, productos o transacciones.

Ejemplo aplicado en un sistema empresarial

En un sistema de inventario es posible observar cómo se combinan los tipos de datos y las estructuras para representar información:

Tipos de datos:

- Precio → numérico
- Nombre del producto → cadena de texto
- Disponible → booleano

Estructura de datos

- Producto = {nombre, precio, cantidad}

En este caso, la estructura “Producto” agrupa varios datos relacionados, lo que permite gestionar de forma organizada la información dentro del sistema.

2.3. Estructuras de selección y estructuras de repetición

En programación, las estructuras de control permiten definir el flujo de ejecución de un programa. Estas estructuras determinan el orden en que se ejecutan las instrucciones y permiten que el sistema tome decisiones o repita acciones según determinadas condiciones.

Existen dos tipos principales de estructuras de control utilizadas en la mayoría de lenguajes de programación: estructuras de selección, que permiten elegir entre diferentes caminos de ejecución, y estructuras de repetición, que permiten ejecutar un conjunto de instrucciones varias veces.

Las estructuras de selección, también llamadas estructuras condicionales, permiten que el programa evalúe una condición y determine qué conjunto de instrucciones debe ejecutarse.

Tabla 7. Estructuras de selección

Estructura	Descripción	Uso común
if	Ejecuta un bloque de instrucciones únicamente si la condición evaluada es verdadera.	Validaciones simples.
if – else	Permite ejecutar un bloque cuando la condición es verdadera y otro cuando es falsa.	Toma de decisiones binarias.
switch / case	Evalúa una variable y permite seleccionar entre múltiples opciones posibles.	Menús o múltiples condiciones.

Estas estructuras son fundamentales para implementar lógica de decisión, validar datos de entrada, controlar accesos o determinar comportamientos dentro de una aplicación.

Las estructuras de repetición, también conocidas como bucles o ciclos, permiten ejecutar un conjunto de instrucciones varias veces mientras se cumpla una condición o durante un número determinado de iteraciones.

Tabla 8. Estructuras de repetición

Estructura	Descripción	Uso común
while	Evalúa la condición antes de ejecutar el bloque de instrucciones. El ciclo continúa mientras la condición sea verdadera.	Procesos cuya repetición depende de una condición.
do – while	Ejecuta el bloque de instrucciones al menos una vez y luego evalúa la condición.	Procesos que requieren ejecutarse mínimo una vez.
for	Se utiliza cuando se conoce previamente el número de iteraciones. Incluye inicialización, condición y actualización.	Recorrer listas o arreglos.

Estas estructuras permiten automatizar tareas repetitivas, recorrer colecciones de datos, procesar registros y ejecutar operaciones múltiples dentro de un programa.

En el desarrollo de aplicaciones web, las estructuras de selección y repetición se utilizan para validar formularios, procesar datos enviados por los usuarios, recorrer listas de información y controlar el comportamiento dinámico de las aplicaciones.

2.4. Funciones, procedimientos, clases y objetos

En el desarrollo de software y aplicaciones web, la organización del código es un aspecto fundamental para garantizar que los programas sean comprensibles, reutilizables y fáciles de mantener. Para lograrlo, los lenguajes de programación incorporan mecanismos que permiten dividir los programas en unidades lógicas de trabajo, como las funciones, los procedimientos y las estructuras propias de la programación orientada a objetos, como las clases y los objetos.

Las funciones y los procedimientos permiten encapsular instrucciones que realizan tareas específicas dentro de un programa. Esta modularización facilita la reutilización del código, evita la repetición innecesaria de instrucciones y permite estructurar los programas de manera más clara.

Por su parte, la programación orientada a objetos introduce el uso de clases y objetos como una forma de representar entidades del mundo real dentro del software. Este enfoque permite organizar los programas a partir de modelos que integran datos y comportamientos, lo que favorece el desarrollo de aplicaciones más estructuradas, escalables y fáciles de mantener, especialmente en entornos de desarrollo web modernos.

Conceptos clave:

- **Funciones:** son bloques de código diseñados para realizar una tarea específica dentro de un programa y que devuelven un valor como resultado de su ejecución. Las funciones se utilizan con frecuencia en cálculos, procesamiento de datos o transformaciones de información.
- **Procedimientos:** son estructuras similares a las funciones, pero su propósito principal es ejecutar una serie de acciones sin devolver un valor como resultado. Se utilizan cuando se requiere realizar operaciones o procesos dentro del programa.
- **Clases:** son estructuras que funcionan como plantillas o modelos para definir las características y comportamientos que tendrán determinados elementos dentro de un programa. En una clase se definen los atributos, que representan datos o propiedades, y los métodos, que representan acciones o comportamientos.

- **Objetos:** son instancias creadas a partir de una clase. Cada objeto representa una entidad concreta que posee valores propios en sus atributos y puede ejecutar los métodos definidos en su clase.

Las funciones y los procedimientos permiten estructurar el código en tareas específicas que pueden reutilizarse en diferentes partes del programa. Mientras las funciones devuelven un valor que puede utilizarse dentro de una expresión o cálculo, los procedimientos se emplean principalmente para ejecutar operaciones o acciones dentro del sistema.

En el caso de la programación orientada a objetos, la clase define la estructura general de una entidad, mientras que el objeto representa una instancia específica creada a partir de esa estructura. Por ejemplo, una clase puede definir las características de un vehículo, mientras que un objeto representaría un vehículo particular con atributos como color, marca o modelo.

El uso de funciones, procedimientos, clases y objetos contribuye a mejorar la organización del código, facilita el mantenimiento del software y promueve la reutilización de componentes en el desarrollo de aplicaciones web.

3. Arquitectura orientada a servicios

Las soluciones tecnológicas actuales rara vez funcionan como sistemas aislados. En entornos empresariales y de computación en la nube es común que diferentes aplicaciones intercambien información y funcionalidades mediante servicios especializados. Este enfoque permite que sistemas desarrollados con distintas tecnologías puedan comunicarse y colaborar entre sí.

En este contexto surge la arquitectura orientada a servicios, un modelo que facilita la integración de aplicaciones mediante componentes reutilizables y autónomos que interactúan a través de redes y protocolos estandarizados. Comprender este enfoque permite interpretar cómo las aplicaciones modernas comparten datos, ejecutan procesos y construyen soluciones escalables y flexibles.

3.1. Conceptos de arquitectura orientada a servicios

La arquitectura orientada a servicios constituye un enfoque de diseño de software que permite estructurar las aplicaciones como un conjunto de servicios independientes que interactúan entre sí a través de una red. Este modelo facilita la integración de sistemas desarrollados con diferentes tecnologías, permitiendo que intercambien información y funcionalidades de manera organizada y eficiente.

La Arquitectura Orientada a Servicios (SOA, por sus siglas en inglés Service-Oriented Architecture) organiza las funcionalidades de negocio en servicios autónomos, reutilizables y modulares. Cada servicio representa una capacidad específica del sistema, como consultar información, procesar datos o ejecutar una operación determinada, y puede ser utilizado por diferentes aplicaciones o plataformas.

Entre las principales características de la arquitectura orientada a servicios se destacan las siguientes:

- **Servicios independientes**

Cada servicio funciona de manera autónoma y puede ejecutarse sin depender directamente de otros componentes del sistema.

- **Reutilización de funcionalidades**

Los servicios pueden ser utilizados por múltiples aplicaciones, evitando la duplicación de código y optimizando el desarrollo.

- **Interoperabilidad**

Permite que sistemas desarrollados en distintos lenguajes o plataformas tecnológicas puedan comunicarse entre sí.

- **Interfaces estandarizadas**

Los servicios se publican mediante interfaces que actúan como un contrato entre el proveedor del servicio y las aplicaciones que lo consumen.

- **Acoplamiento flexible**

Las aplicaciones pueden interactuar con los servicios sin conocer los detalles internos de su implementación.

En muchas implementaciones de SOA se emplea un bus de servicios empresariales (ESB), que funciona como un componente intermediario encargado de gestionar la comunicación entre los diferentes servicios. Este elemento facilita tareas

como el enrutamiento de mensajes, la transformación de datos y la conexión entre aplicaciones dentro de una infraestructura tecnológica.

Este enfoque permite construir sistemas más flexibles, escalables y fáciles de mantener. A partir de estos principios se desarrollan diferentes tecnologías y protocolos para implementar servicios web, entre los cuales se destacan SOAP y REST.

3.2. Servicios web SOAP

SOAP (Simple Object Access Protocol) es un protocolo utilizado para el intercambio de información entre aplicaciones a través de la red. Su objetivo es permitir que diferentes sistemas informáticos se comuniquen entre sí, incluso si están desarrollados en lenguajes de programación o plataformas tecnológicas distintas. Este protocolo es ampliamente utilizado en entornos empresariales donde se requiere una comunicación estructurada, segura y confiable entre servicios.

Los servicios web basados en SOAP utilizan XML (Extensible Markup Language) como formato estándar para estructurar los mensajes que se envían entre aplicaciones. Estos mensajes suelen transmitirse mediante protocolos de red como HTTP, lo que permite que los servicios se integren fácilmente con la infraestructura de Internet.

Un mensaje SOAP está compuesto por diferentes elementos que permiten organizar la información y definir cómo debe procesarse. Entre sus componentes principales se encuentran:

- **Sobre (Envelope)**

Es el contenedor principal del mensaje SOAP. Define la estructura del documento y especifica cómo debe interpretarse el mensaje.

- **Cabecera (Header)**

Contiene información adicional de control, como aspectos relacionados con seguridad, autenticación o calidad del servicio.

- **Cuerpo (Body)**

Incluye los datos o la solicitud que se envía al servicio, así como los parámetros necesarios para ejecutar una operación específica.

En los servicios SOAP también se utilizan contratos formales de servicio, comúnmente descritos mediante WSDL (Web Services Description Language). Este lenguaje permite definir la estructura del servicio, las operaciones disponibles, los parámetros requeridos y la forma en que las aplicaciones pueden interactuar con él.

Entre las características más importantes de los servicios web SOAP se destacan las siguientes:

- Mensajería estructurada basada en XML, que garantiza una comunicación clara entre aplicaciones.
- Interoperabilidad entre diferentes plataformas y lenguajes de programación.
- Uso de contratos formales de servicio, que describen de manera precisa cómo debe consumirse el servicio.

- Alto nivel de estandarización, lo que facilita su adopción en entornos empresariales.

Debido a su estructura formal y a sus mecanismos de seguridad integrados, SOAP ha sido ampliamente utilizado en sistemas corporativos y en procesos de integración entre aplicaciones empresariales. Sin embargo, en aplicaciones web modernas también se utilizan otros enfoques de comunicación más ligeros, como los servicios basados en REST, que se abordarán en el siguiente apartado.

3.3. Servicios web REST

REST (Representational State Transfer o Transferencia de Estado Representacional) es un estilo arquitectónico utilizado para diseñar servicios web que permiten la comunicación entre aplicaciones a través de Internet. A diferencia de SOAP, que es un protocolo formal, REST se basa en un conjunto de principios que orientan la forma en que los sistemas intercambian información mediante el uso de recursos accesibles a través de la web.

En los servicios REST, cada recurso como un usuario, un producto o una orden se identifica mediante una URL, y las aplicaciones interactúan con estos recursos utilizando métodos estándar del protocolo HTTP, como GET, POST, PUT y DELETE. De esta manera, los servicios REST facilitan la creación de aplicaciones web flexibles, escalables y fáciles de integrar con distintos tipos de sistemas.

Una característica importante de REST es que las solicitudes realizadas al servidor son sin estado (stateless), lo que significa que cada petición contiene toda la información necesaria para ser procesada, sin depender de solicitudes anteriores. Esto

permite que los sistemas sean más eficientes y escalables, especialmente en entornos de computación en la nube.

Los servicios web REST presentan varias características que favorecen su adopción en el desarrollo web moderno:

- Uso de estándares web ampliamente adoptados, como HTTP y URL.
- Comunicación sin estado, lo que mejora la escalabilidad de las aplicaciones.
- Intercambio de información mediante formatos ligeros, principalmente JSON, aunque también pueden utilizarse XML, HTML o texto plano.
- Facilidad de integración con aplicaciones web, móviles y servicios en la nube.

Gracias a su simplicidad y flexibilidad, los servicios REST son ampliamente utilizados en el desarrollo de APIs (Interfaces de Programación de Aplicaciones) que permiten que diferentes sistemas compartan información y funcionalidades. Este enfoque es especialmente común en aplicaciones modernas, plataformas de servicios digitales, microservicios y entornos de computación en la nube.

En comparación con SOAP, los servicios REST suelen ser más ligeros y fáciles de implementar, lo que ha favorecido su adopción en aplicaciones web contemporáneas y en servicios que requieren alta escalabilidad y rendimiento.

3.4. Aplicaciones y almacenamiento de datos en arquitecturas de servicios

En las arquitecturas orientadas a servicios, las aplicaciones se diseñan como un conjunto de componentes independientes que se comunican entre sí mediante servicios o APIs. Este enfoque permite que cada servicio cumpla una función específica dentro del sistema, facilitando el desarrollo, la escalabilidad y el mantenimiento de las aplicaciones.

En este tipo de arquitectura, es común que cada servicio gestione sus propios datos o recursos de almacenamiento. De esta manera, se reduce la dependencia entre componentes y se permite que cada parte del sistema evolucione de forma independiente. Además, en muchos entornos actuales estas soluciones se implementan sobre infraestructuras en la nube, lo que proporciona mayor disponibilidad, flexibilidad y capacidad de crecimiento según las necesidades del sistema.

Entre los componentes más utilizados en arquitecturas basadas en servicios se encuentran los siguientes:

- **Bases de datos como servicio (DBaaS)**

Permiten utilizar bases de datos administradas en la nube sin necesidad de gestionar directamente la infraestructura. Facilitan tareas como copias de seguridad, escalado automático y mantenimiento del sistema de base de datos.

- **Almacenamiento como servicio (STaaS)**

Se utiliza para guardar grandes volúmenes de información no estructurada, como archivos, imágenes, videos o copias de seguridad. Este tipo de almacenamiento ofrece alta disponibilidad y acceso desde diferentes aplicaciones o servicios.

- **Plataformas de análisis de datos**

Herramientas como data warehouse o data lakes permiten concentrar grandes cantidades de información para su análisis, generación de reportes e inteligencia de negocio.

- **API Gateway**

Es un componente que actúa como punto de entrada para las solicitudes que realizan las aplicaciones cliente. Su función consiste en recibir las peticiones, dirigirlas al servicio correspondiente y gestionar aspectos como seguridad, autenticación o control del tráfico.

La integración de estos componentes permite construir sistemas más flexibles, resilientes y preparados para entornos distribuidos, donde múltiples aplicaciones interactúan entre sí mediante servicios web o APIs.

4. Desarrollo de APIs y configuración del entorno

El desarrollo de APIs constituye uno de los ejes centrales en la construcción de soluciones basadas en la nube y en arquitecturas modernas de software. Una API bien diseñada permite conectar aplicaciones, exponer funcionalidades del sistema, sincronizar datos y automatizar procesos entre diferentes plataformas internas y externas. Por esta razón, su implementación requiere un entorno de desarrollo adecuado, herramientas especializadas y buenas prácticas desde las primeras etapas del diseño.

Una API, o interfaz de programación de aplicaciones, es un conjunto de reglas, protocolos y definiciones que permite que diferentes aplicaciones de software se comuniquen entre sí e intercambien información. Actúa como un intermediario que facilita el acceso a determinadas funcionalidades o recursos de un sistema sin que las aplicaciones que la utilizan necesiten conocer los detalles internos de su implementación.

De esta manera, las APIs establecen cómo deben realizarse las solicitudes de información, qué parámetros se deben enviar y qué tipo de respuesta se puede recibir, garantizando una comunicación estructurada y segura entre los distintos componentes de una aplicación o entre sistemas desarrollados por diferentes organizaciones.

4.1. Preparación del entorno de desarrollo

La preparación del entorno de desarrollo es un paso fundamental antes de iniciar la construcción de una API. Un entorno correctamente configurado permite organizar el

código, facilitar las pruebas, gestionar dependencias y asegurar que la aplicación funcione de manera adecuada durante todo el proceso de desarrollo.

En el contexto del desarrollo de APIs, el entorno de trabajo incluye herramientas para la escritura de código, ejecución del lenguaje de programación, gestión de bases de datos, control de versiones y pruebas de los servicios. Estas herramientas permiten a los desarrolladores crear, probar y documentar las interfaces de manera organizada y eficiente.

Entre los elementos más comunes que conforman un entorno de desarrollo para APIs se encuentran:

- **Entorno de desarrollo integrado (IDE)**

Son aplicaciones que facilitan la escritura, depuración y organización del código. Entre los más utilizados se encuentran Visual Studio Code, Visual Studio o IntelliJ IDEA.

- **Lenguaje y entorno de ejecución**

Dependiendo de la tecnología utilizada, es necesario instalar el entorno correspondiente, como Node.js para aplicaciones JavaScript, Python para desarrollo con frameworks como Flask o Django, o .NET para aplicaciones basadas en el ecosistema de Microsoft.

- **Gestión de bases de datos**

Muchas APIs interactúan con sistemas de almacenamiento de información. Por ello, se suelen configurar bases de datos como MySQL, PostgreSQL o MongoDB, junto con herramientas de administración que facilitan su manejo.

- **Herramientas de pruebas de API**

Aplicaciones como Postman o Insomnia permiten enviar solicitudes HTTP y analizar las respuestas del servidor, lo que facilita la verificación del funcionamiento de los servicios desarrollados.

- **Control de versiones**

Sistemas como Git permiten registrar los cambios realizados en el código, facilitar el trabajo colaborativo entre desarrolladores y mantener un historial de modificaciones del proyecto.

- **Documentación de la API**

Herramientas como Swagger u OpenAPI permiten describir la estructura de la API, los endpoints disponibles, los parámetros requeridos y los tipos de respuesta, facilitando su comprensión y uso por parte de otros desarrolladores.

Además, es recomendable separar los entornos de desarrollo, pruebas y producción, con el fin de evitar que los cambios realizados durante la programación afecten directamente a las aplicaciones en funcionamiento.

4.2. Requerimientos de hardware y software de IDE

El entorno de desarrollo integrado o Integrated Development Environment (IDE) constituye una herramienta fundamental para la programación y el desarrollo de aplicaciones. Estos entornos reúnen en una misma plataforma diversas funcionalidades como el editor de código, herramientas de depuración, compiladores, gestores de

dependencias y utilidades de control de versiones, lo que facilita el proceso de desarrollo y mejora la productividad del programador.

Para trabajar de manera eficiente con IDE modernos utilizados en el desarrollo de aplicaciones y APIs, es necesario contar con ciertos requerimientos de hardware y software que permitan ejecutar las herramientas de forma fluida, especialmente cuando se utilizan múltiples servicios, bases de datos, navegadores y herramientas de prueba de manera simultánea.

A continuación, se presentan los requerimientos de hardware recomendados. Estos componentes permiten trabajar con IDE modernos de forma fluida:

- **Memoria RAM**

Se recomienda un mínimo de 8 GB para proyectos sencillos o desarrollo web básico. Para proyectos más complejos o cuando se utilizan múltiples herramientas simultáneamente, se recomienda contar con 16 GB o más.

- **Procesador (CPU)**

Un procesador de gama media o superior permite ejecutar el IDE, compilar código y realizar tareas de depuración con mayor rapidez. Procesadores multinúcleo mejoran el rendimiento en procesos de compilación y ejecución.

- **Almacenamiento**

Es recomendable utilizar unidades de estado sólido (SSD), ya que permiten una carga más rápida del sistema operativo, del IDE y de los proyectos de desarrollo. Un espacio de almacenamiento de al menos 256 GB suele ser suficiente para proyectos académicos o iniciales.

- **Pantalla y resolución**

Una resolución de al menos 1920 × 1080 píxeles, facilita la visualización simultánea de múltiples ventanas de código, consolas y herramientas de depuración.

En relación con los requerimientos de software, además del hardware es necesario disponer de un entorno adecuado que permita instalar y ejecutar las herramientas de desarrollo:

- **Sistema operativo**

Los IDE modernos funcionan generalmente en sistemas operativos como Windows, macOS o distribuciones Linux actualizadas.

- **Entornos de ejecución y dependencias**

Dependiendo del lenguaje utilizado, pueden requerirse componentes adicionales como el Java Development Kit (JDK), Node.js, Python o entornos de ejecución específicos del lenguaje.

- **Gestores de dependencias**

Herramientas como npm, Maven, Gradle o pip permiten instalar librerías y paquetes necesarios para el desarrollo de aplicaciones.

- **Conexión a Internet**

Es necesaria para descargar plugins, dependencias, actualizaciones del IDE y bibliotecas utilizadas en los proyectos.

A continuación, se presenta una comparación general de los requerimientos según el nivel de trabajo.

Tabla 9. Comparativa de requerimientos según el nivel de uso

Componente	Nivel básico (aprendiz)	Nivel avanzado o profesional
Memoria RAM.	8 GB.	16 GB o más.
Procesador.	Gama media.	Gama media-alta o superior.
Almacenamiento.	256 GB SSD.	512 GB SSD o más.
Gráficos.	Integrados.	Integrados o dedicados según el proyecto.

En conjunto, estos requerimientos permiten disponer de un entorno de desarrollo estable y eficiente para la construcción de aplicaciones y APIs. Contar con un equipo y un entorno de software adecuados facilita la ejecución de herramientas modernas, la gestión de dependencias y la integración con servicios en la nube, aspectos que resultan fundamentales en los procesos actuales de desarrollo de software.

4.3. Codificación de módulos y desarrollo de APIs

En el desarrollo de aplicaciones modernas, la codificación de módulos permite organizar el sistema en componentes independientes que cumplen funciones específicas. Esta modularidad facilita el mantenimiento del software, mejora la reutilización del código y permite que distintos equipos trabajen de manera paralela sobre diferentes partes del sistema.

En el contexto de las arquitecturas basadas en servicios, estos módulos suelen implementarse mediante APIs (Application Programming Interface). Una API es un

conjunto de reglas que permite que diferentes aplicaciones se comuniquen entre sí y compartan información o funcionalidades sin necesidad de conocer el funcionamiento interno de cada sistema.

Existen diferentes estilos de diseño de APIs, siendo uno de los más utilizados el modelo REST (Representational State Transfer). Las APIs REST permiten que las aplicaciones intercambien información a través del protocolo HTTP, utilizando direcciones web (URL) para identificar los recursos y métodos estándar para realizar operaciones sobre ellos.

El funcionamiento básico de una API REST sigue un modelo de comunicación cliente-servidor. Cuando una aplicación cliente requiere acceder a un recurso, envía una solicitud al servidor mediante la API, y este procesa la petición y devuelve una respuesta con la información solicitada o el resultado de la operación.

De manera general, el proceso de interacción con una API REST se desarrolla en las siguientes etapas:

- El cliente envía una solicitud al servidor siguiendo el formato definido en la documentación de la API.
- El servidor autentica al cliente y verifica que tenga permisos para realizar la operación.
- El servidor procesa la solicitud internamente mediante la lógica de negocio implementada en los módulos del sistema.
- Finalmente, el servidor devuelve una respuesta que indica si la operación se realizó correctamente y, cuando corresponde, incluye los datos solicitados.

Esta estructura permite construir aplicaciones escalables, ya que los diferentes módulos pueden evolucionar de forma independiente y comunicarse mediante interfaces claramente definidas.

Para comprender el funcionamiento de una API REST, se puede considerar el caso de un sistema de gestión de productos en una tienda en línea. En este sistema, cada producto representa un recurso accesible mediante una URL.

Por ejemplo:

- GET /productos permite consultar la lista de productos disponibles.
- GET /productos/25 permite consultar la información de un producto específico.
- POST /productos permite registrar un nuevo producto en el sistema.
- PUT /productos/25 permite actualizar la información de un producto existente.
- DELETE /productos/25 permite eliminar un producto del sistema.

Este tipo de estructura facilita la integración entre aplicaciones y permite que diferentes clientes, como aplicaciones web o móviles, interactúen con el mismo servicio de manera estandarizada.

4.4. Solicitudes, respuestas y autenticación en APIs REST

En las APIs REST, la comunicación entre aplicaciones se realiza mediante solicitudes enviadas por un cliente y respuestas generadas por un servidor. Este modelo

permite que diferentes sistemas intercambien información de forma estandarizada utilizando el protocolo HTTP.

Cuando una aplicación cliente necesita acceder a un recurso o ejecutar una operación, envía una solicitud a la API. El servidor procesa dicha solicitud y devuelve una respuesta que indica si la operación se realizó correctamente y, en caso necesario, incluye los datos solicitados.

- **Elementos de una solicitud en una API REST**

Las solicitudes enviadas por el cliente al servidor suelen incluir varios componentes que permiten identificar el recurso solicitado y la operación que se desea realizar.

- a) **Identificador del recurso (URL):** cada recurso disponible en la API se identifica mediante una dirección única denominada URL. Esta dirección indica al servidor qué recurso desea consultar o manipular el cliente. Por ejemplo, en una API de productos, la dirección /productos puede utilizarse para consultar la lista de productos disponibles.
- b) **Método HTTP:** indica la acción que el cliente desea realizar sobre el recurso. Los métodos más utilizados permiten consultar, crear, actualizar o eliminar información dentro del sistema.

Tabla 10. Métodos HTTP comunes en APIs REST

Método HTTP	Uso típico	Observación
GET	Consultar recursos.	No debe modificar el estado del servidor.

Método HTTP	Uso típico	Observación
POST	Crear recursos.	Suele generar nuevos registros.
PUT	Actualizar un recurso completo.	Es idempotente cuando se usa correctamente.
DELETE	Eliminar recursos.	Debe validarse mediante autenticación.

- c) Encabezados de la solicitud:** contienen metadatos que describen la solicitud. Estos pueden indicar, por ejemplo, el formato de los datos enviados, el tipo de respuesta esperado o información relacionada con la autenticación del cliente.
- d) Datos o cuerpo de la solicitud:** algunos métodos, como POST o PUT, permiten enviar información adicional al servidor dentro del cuerpo de la solicitud. Estos datos suelen representarse en formatos estructurados como JSON o XML, lo que facilita su procesamiento por parte de las aplicaciones.
- e) Parámetros:** las solicitudes también pueden incluir parámetros que permiten especificar detalles adicionales sobre la operación que se desea realizar. Entre los más comunes se encuentran:
- Parámetros de ruta, que forman parte de la URL y permiten identificar recursos específicos.
 - Parámetros de consulta, que agregan información adicional a la solicitud, como filtros o criterios de búsqueda.

- Parámetros de cookie, utilizados en algunos casos para mantener información de sesión.
- Elementos de la respuesta del servidor

Una vez que el servidor procesa la solicitud del cliente, envía una respuesta que indica el resultado de la operación. Esta respuesta suele incluir los siguientes componentes:

- a) **Línea de estado:** contiene un código de tres dígitos que informa si la solicitud se procesó correctamente o si ocurrió algún error durante la operación. Algunos códigos de estado comunes son:
 - 200: la solicitud se procesó correctamente.
 - 201: el recurso fue creado exitosamente.
 - 400: la solicitud es incorrecta o contiene errores.
 - 404: el recurso solicitado no fue encontrado.
- b) **Cuerpo del mensaje:** el cuerpo de la respuesta contiene la representación del recurso solicitado o el resultado de la operación. Esta información suele enviarse en formatos estructurados como JSON o XML.
- c) **Encabezados de la respuesta:** los encabezados proporcionan información adicional sobre la respuesta, como el tipo de contenido, la fecha de la respuesta, el servidor que procesó la solicitud o las políticas de seguridad aplicadas.

- Métodos de autenticación en APIs REST

Para proteger la información y garantizar que solo los usuarios autorizados puedan acceder a los recursos, las APIs REST utilizan diferentes mecanismos de autenticación.

Tabla 11. Formas comunes de autenticación para servicios REST

Método de autenticación	Descripción
Básica	Envía usuario y contraseña codificados en el encabezado de la solicitud.
Bearer Token	Utiliza un token generado por el servidor para autorizar el acceso.
API Key	Asigna una clave única al cliente para validar el acceso a los recursos.
OAuth	Sistema de autenticación y autorización más robusto basado en tokens.

La implementación adecuada de estos mecanismos de autenticación permite proteger los servicios, controlar el acceso a los recursos y garantizar la integridad de la información intercambiada entre las aplicaciones.

En entornos de desarrollo cloud, es recomendable complementar la codificación de APIs con prácticas como documentación técnica, pruebas automatizadas, control de versiones y monitoreo del servicio. Estas estrategias contribuyen a mejorar la mantenibilidad, la seguridad y la escalabilidad de las aplicaciones basadas en servicios.

4.5. Migraciones y sincronización de bases de datos

En el desarrollo de aplicaciones, especialmente en entornos cloud o arquitecturas basadas en servicios, es común que la estructura de las bases de datos evolucione con el tiempo. A medida que el software crece, se agregan nuevas tablas, se modifican campos o se reorganizan los datos para mejorar el rendimiento y la escalabilidad.

Para gestionar estos cambios de forma segura se utilizan migraciones y procesos de sincronización, los cuales permiten actualizar la estructura de la base de datos y mantener la coherencia de la información entre distintos sistemas o entornos.

Las migraciones de bases de datos consisten en un conjunto de instrucciones o scripts que permiten crear, modificar o eliminar estructuras de datos de forma controlada. Estas migraciones suelen almacenarse como archivos versionados que pueden aplicarse o revertirse cuando sea necesario, lo que facilita el mantenimiento y la evolución del sistema.

Por su parte, la sincronización de datos permite mantener consistentes diferentes bases de datos o servicios que comparten información. Este proceso es especialmente importante cuando existen múltiples aplicaciones conectadas o cuando se realizan procesos de migración hacia infraestructuras en la nube.

Entre los enfoques más utilizados para gestionar migraciones y sincronización de datos se encuentran los siguientes:

- **Migraciones estructurales**

Permiten definir cambios en el esquema de la base de datos, como la creación de tablas, la modificación de columnas o la eliminación de estructuras obsoletas. Estas

migraciones se gestionan mediante archivos versionados que pueden aplicarse de forma secuencial.

- **Migración de datos (ETL)**

Consiste en el proceso de extraer, transformar y cargar datos entre diferentes sistemas o motores de base de datos. Este enfoque se utiliza cuando es necesario trasladar información desde una plataforma a otra.

- **Captura de datos modificados (CDC)**

Es una técnica que permite identificar y replicar los cambios realizados en una base de datos en tiempo casi real. Esto facilita mantener sistemas sincronizados sin necesidad de detener las operaciones del sistema principal.

- **Sincronización mediante APIs**

En arquitecturas basadas en servicios, las APIs permiten intercambiar información entre diferentes aplicaciones o plataformas externas, garantizando que los datos se mantengan actualizados en todos los sistemas conectados.

Tabla 12. Enfoques frecuentes de migración y sincronización de datos

Enfoque	Finalidad
Migración estructural	Aplicar cambios controlados en el esquema de la base de datos.
ETL	Extraer, transformar y cargar datos entre sistemas o motores de base de datos.
CDC	Replicar cambios en tiempo casi real para mantener la consistencia de la información.
Sincronización por APIs	Intercambiar datos entre aplicaciones o plataformas externas.

La correcta planificación de estos procesos permite evitar pérdida de información, garantizar la integridad de los datos y asegurar la continuidad operativa del sistema. Por esta razón, las migraciones suelen realizarse siguiendo buenas prácticas como la creación de copias de seguridad, la ejecución controlada de scripts y la validación posterior de los datos migrados.

4.6. ORM (Object Relational Mapping)

El mapeo objeto-relacional, conocido como ORM (Object Relational Mapping), es una técnica de programación que permite conectar el modelo orientado a objetos de una aplicación con la estructura tabular de una base de datos relacional. Su propósito es facilitar la interacción con los datos utilizando objetos del lenguaje de programación en lugar de escribir consultas SQL de forma manual.

En el desarrollo de aplicaciones modernas, el código suele organizarse mediante clases, objetos y métodos, mientras que las bases de datos relacionales almacenan la información en tablas, filas y columnas. El ORM actúa como una capa de traducción entre estos dos modelos, permitiendo que los desarrolladores manipulen datos mediante objetos y que el sistema genere automáticamente las consultas SQL necesarias.

Gracias a esta técnica, una clase del programa puede representar una tabla de la base de datos, mientras que los atributos de la clase se relacionan con las columnas de dicha tabla. De esta manera, cada instancia del objeto representa un registro dentro de la base de datos.

El funcionamiento de un ORM se basa en el mapeo entre las estructuras del código y las estructuras de la base de datos. Cuando una aplicación realiza una operación sobre un objeto, el ORM traduce automáticamente esa acción en una consulta SQL equivalente.

Entre las principales funciones que realiza un ORM se encuentran:

- **Mapeo entre clases y tablas:** permite que las clases del código representen tablas dentro de la base de datos, estableciendo correspondencia entre atributos y columnas.
- **Automatización de operaciones CRUD:** las operaciones de crear, leer, actualizar y eliminar registros pueden realizarse mediante métodos del lenguaje de programación, sin necesidad de escribir consultas SQL manuales.
- **Gestión de relaciones entre entidades:** permite representar relaciones entre tablas, como uno a uno, uno a muchos o muchos a muchos, directamente a través de objetos y colecciones dentro del código.
- **Gestión de la persistencia de datos:** controla el almacenamiento, actualización y recuperación de la información desde la base de datos, manteniendo sincronizados los objetos del sistema con los registros persistentes.

Gracias a estas capacidades, el ORM permite que el desarrollador se concentre en la lógica del negocio de la aplicación sin preocuparse constantemente por los detalles de las consultas SQL.

En este contexto, existen herramientas de ORM disponibles para distintos lenguajes de programación. Estas herramientas se implementan mediante diversas librerías diseñadas para integrarse con los principales entornos de desarrollo y facilitar la interacción entre el código de la aplicación y la base de datos. Algunos ejemplos son:

- **Python:** SQLAlchemy y Django ORM.
- **Java:** Hibernate y JPA.
- **C# / .NET:** Entity Framework Core y Dapper.
- **JavaScript (Node.js):** Sequelize, Prisma y TypeORM.
- **PHP:** Eloquent (Laravel) y Doctrine.

Cada una de estas herramientas implementa el concepto de mapeo objeto-relacional con diferentes enfoques, pero todas comparten el objetivo de simplificar la interacción entre aplicaciones y bases de datos.

Tabla 13. Ventajas y consideraciones del uso de ORM

Característica	Descripción
Productividad.	Reduce la cantidad de código repetitivo necesario para interactuar con la base de datos.
Seguridad.	Disminuye el riesgo de ataques de inyección SQL mediante consultas parametrizadas.
Portabilidad.	Facilita el cambio de motor de base de datos con ajustes mínimos en el código.
Rendimiento.	En consultas muy complejas puede resultar menos eficiente que escribir SQL optimizado manualmente.

El uso adecuado de ORM mejora la productividad en el desarrollo de software, especialmente en aplicaciones con arquitecturas por capas, APIs y sistemas que manejan grandes volúmenes de datos. Sin embargo, es importante que los desarrolladores comprendan el funcionamiento de la base de datos subyacente, ya que una mala configuración del ORM puede generar consultas ineficientes o problemas de rendimiento.

5. Pruebas en el desarrollo de APIs

Las pruebas de software constituyen una etapa fundamental dentro del proceso de desarrollo de aplicaciones y servicios digitales. En el caso de las APIs, las pruebas permiten verificar que los servicios expuestos funcionen correctamente, respondan de forma consistente a las solicitudes de los clientes y manejen adecuadamente situaciones de error.

Una API actúa como un punto de comunicación entre diferentes sistemas o aplicaciones. Por esta razón, cualquier fallo en su funcionamiento puede afectar múltiples componentes de un sistema o incluso a aplicaciones externas que dependen de ella. Las pruebas permiten detectar estos problemas antes de que el servicio sea puesto en producción, reduciendo riesgos operativos y mejorando la calidad del software.

En el desarrollo de APIs, las pruebas se enfocan en validar distintos aspectos del servicio, como el correcto procesamiento de solicitudes, la precisión de las respuestas generadas por el servidor, el cumplimiento de los contratos definidos en la documentación de la API y la seguridad en el acceso a los recursos.

Durante este proceso se comprueba, por ejemplo, que los métodos HTTP funcionen según lo esperado, que los parámetros enviados por el cliente sean interpretados correctamente y que los códigos de estado devueltos por el servidor representen adecuadamente el resultado de cada operación.

Además, las pruebas permiten garantizar que la API mantenga un comportamiento estable incluso cuando recibe múltiples solicitudes, cuando se

presentan datos inválidos o cuando se producen condiciones inesperadas dentro del sistema.

En entornos de desarrollo modernos, especialmente en arquitecturas basadas en microservicios o aplicaciones en la nube, las pruebas de APIs suelen integrarse dentro de procesos automatizados de integración y entrega continua (CI/CD). Esto permite validar de forma constante la calidad del servicio cada vez que se realizan cambios en el código.

En consecuencia, la implementación de pruebas sistemáticas no solo mejora la confiabilidad del sistema, sino que también facilita el mantenimiento, la evolución del software y la detección temprana de errores durante el ciclo de desarrollo.

5.1. Conceptos de pruebas de software

Las pruebas de software constituyen una etapa fundamental dentro del proceso de desarrollo de aplicaciones y APIs. Su propósito es verificar que el sistema funcione correctamente, cumpla con los requisitos definidos y responda adecuadamente ante diferentes condiciones de uso.

En el desarrollo de APIs, las pruebas permiten comprobar que los servicios responden de forma correcta a las solicitudes realizadas por los clientes, que los datos se procesan adecuadamente y que la comunicación entre sistemas se mantiene estable y segura. A través de estas verificaciones es posible detectar errores antes de que el software sea utilizado en entornos reales, lo que contribuye a mejorar la calidad, la confiabilidad y el mantenimiento del sistema.

Para comprender el proceso de pruebas, es importante conocer algunos conceptos fundamentales:

- **Prueba de software:** proceso sistemático mediante el cual se evalúa el funcionamiento de un sistema informático para verificar que cumple con los requisitos funcionales y técnicos definidos durante su desarrollo.
- **Error:** equivocación cometida por una persona durante las fases de diseño, programación o configuración del sistema. Un error puede originar defectos en el software.
- **Defecto o bug:** problema presente en el código o en la lógica del sistema que provoca un comportamiento incorrecto del software.
- **Fallo:** manifestación visible de un defecto cuando el sistema se ejecuta y produce un resultado inesperado o incorrecto.
- **Caso de prueba:** conjunto de condiciones, datos y pasos definidos para verificar si una funcionalidad específica del sistema produce el resultado esperado.
- **Cobertura de pruebas:** indicador que permite determinar qué proporción del sistema o del código ha sido evaluada mediante pruebas.

Comprender estos conceptos permite interpretar adecuadamente los resultados obtenidos durante el proceso de verificación del software y facilita la aplicación de diferentes tipos de pruebas que contribuyen a garantizar la calidad de una API.

5.2. Características y tipos de pruebas

Para evaluar adecuadamente el funcionamiento de una API no es suficiente realizar una única verificación. Es necesario aplicar diferentes tipos de pruebas que permitan validar la funcionalidad, el rendimiento, la seguridad y la interacción con otros sistemas.

Cada tipo de prueba cumple un propósito específico dentro del proceso de aseguramiento de calidad del software. La combinación de estas pruebas permite detectar errores, mejorar la estabilidad del sistema y garantizar que los servicios funcionen correctamente en distintos escenarios de uso.

Entre las principales pruebas utilizadas en el desarrollo de APIs se encuentran las siguientes:

- **Pruebas funcionales:** verifican que los endpoints de la API respondan correctamente a las solicitudes realizadas por los clientes. Estas pruebas validan que los códigos de estado, los formatos de datos y la información retornada coincidan con lo especificado en el diseño de la API.
- **Pruebas unitarias:** evalúan componentes individuales del sistema, como funciones o métodos específicos. En el caso de las APIs, permiten comprobar el comportamiento de un endpoint de manera aislada, facilitando la detección temprana de errores en el código.
- **Pruebas de integración:** analizan la interacción entre diferentes componentes del sistema, como servicios, bases de datos o APIs externas. Su objetivo es comprobar que el intercambio de datos y la comunicación entre módulos se realicen correctamente.

- **Pruebas de rendimiento:** miden la capacidad de respuesta, la velocidad y el comportamiento de la API bajo diferentes niveles de carga. Permiten identificar cuellos de botella y evaluar cómo responde el sistema ante múltiples solicitudes simultáneas.
- **Pruebas de carga y estrés:** las pruebas de carga evalúan el rendimiento de la API bajo un volumen determinado de solicitudes concurrentes, mientras que las pruebas de estrés buscan determinar el límite máximo que el sistema puede soportar antes de fallar.
- **Pruebas de seguridad:** se enfocan en identificar vulnerabilidades que puedan comprometer la integridad de los datos o permitir accesos no autorizados. Incluyen la verificación de mecanismos de autenticación, autorización y protección frente a ataques como inyección SQL o manipulación de datos.
- **Pruebas de interoperabilidad:** comprueban que la API pueda comunicarse correctamente con distintos sistemas, plataformas, lenguajes de programación y formatos de datos, garantizando su compatibilidad en diversos entornos tecnológicos.
- **Pruebas de contrato:** validan que la comunicación entre diferentes servicios o microservicios cumpla con los acuerdos establecidos en la especificación de la API, evitando problemas de integración entre sistemas.
- **Pruebas de extremo a extremo:** evalúan el funcionamiento completo del sistema, simulando escenarios reales en los que la API interactúa con bases de datos, servicios externos y aplicaciones cliente.

Para facilitar la comprensión de estos enfoques, en la siguiente tabla se presentan algunos de los tipos de pruebas más relevantes en el desarrollo de APIs.

Tabla 14. Tipos de pruebas relevantes en el desarrollo de APIs

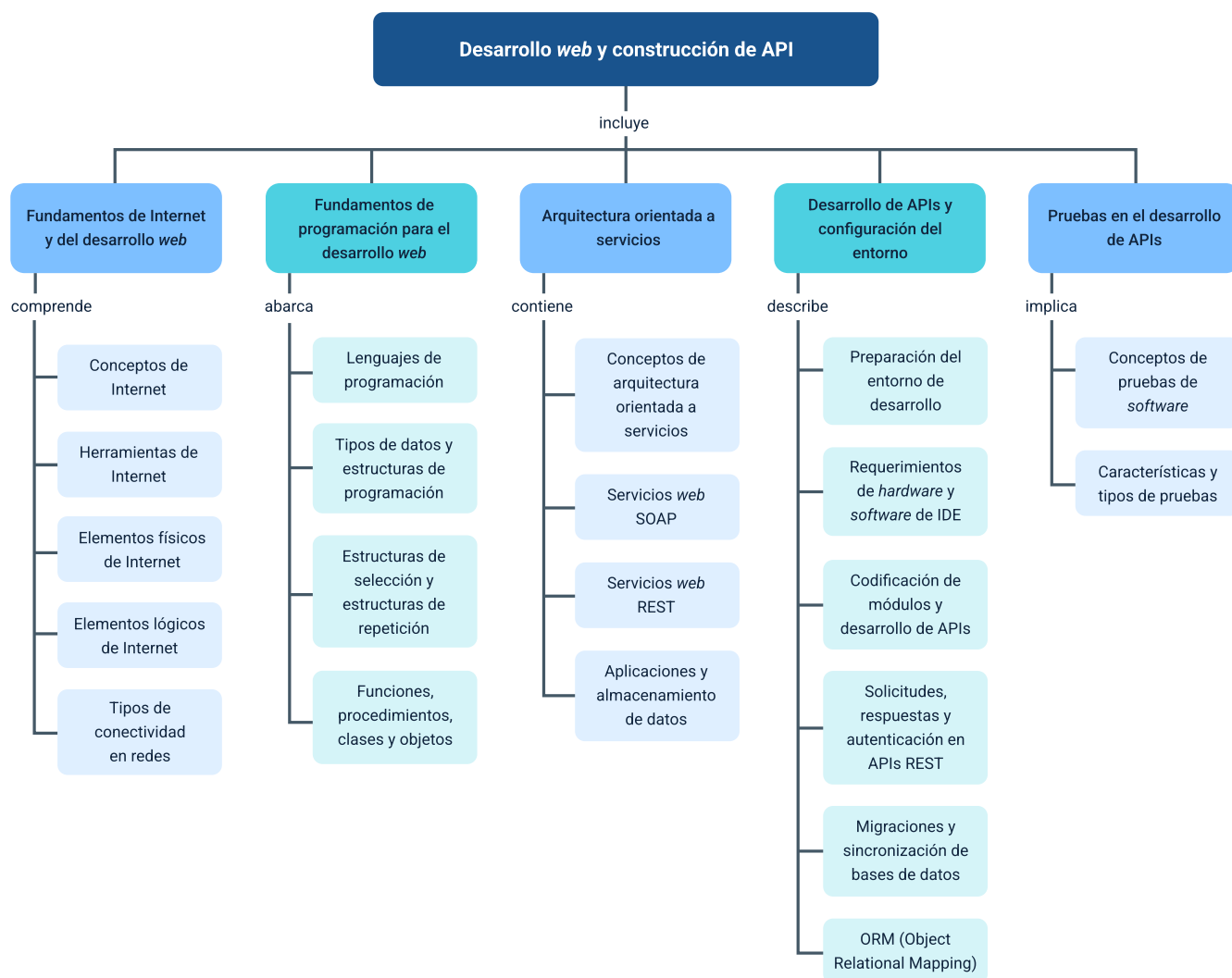
Tipo de prueba	Finalidad principal
Funcional	Verificar que los endpoints respondan de acuerdo con la especificación.
Seguridad	Detectar vulnerabilidades en autenticación, autorización y manejo de datos.
Rendimiento	Evaluar la velocidad y capacidad de respuesta del servicio.
Carga / estrés	Medir el comportamiento bajo diferentes niveles de tráfico.
Integración	Validar la interacción entre servicios y sistemas externos.
Regresión	Confirmar que cambios recientes no afectaron funcionalidades existentes.
Extremo a extremo	Evaluar el flujo completo de funcionamiento del sistema.

En la práctica, las pruebas pueden realizarse de forma manual o automatizada. Las pruebas manuales son útiles en etapas iniciales o para explorar el comportamiento de la API, mientras que las pruebas automatizadas permiten ejecutar validaciones repetitivas de forma rápida y confiable, especialmente en entornos de integración continua.

La aplicación sistemática de estas pruebas contribuye a garantizar APIs más estables, seguras y confiables dentro de los sistemas de información modernos.

Síntesis

El componente formativo integra los fundamentos de Internet y del desarrollo web, los principios de programación utilizados para construir aplicaciones, la arquitectura orientada a servicios y los procesos técnicos necesarios para desarrollar APIs. A lo largo del componente se abordan aspectos relacionados con la configuración del entorno de desarrollo, la comunicación entre aplicaciones mediante solicitudes y respuestas, la gestión de datos y la realización de pruebas que permiten verificar el funcionamiento de los servicios. Estos conocimientos proporcionan una base conceptual y técnica para comprender cómo se diseñan, construyen y validan aplicaciones y servicios digitales que interactúan a través de la web.



Glosario

API (Interfaz de Programación de Aplicaciones): conjunto de reglas, protocolos y definiciones que permiten que diferentes aplicaciones o sistemas se comuniquen e intercambien información de forma estructurada.

Arquitectura orientada a servicios (SOA): modelo de diseño de software en el que las funcionalidades de una aplicación se organizan como servicios independientes que pueden ser utilizados por distintos sistemas a través de una red.

Autenticación: proceso mediante el cual un sistema verifica la identidad de un usuario o aplicación antes de permitir el acceso a recursos o servicios.

Endpoint: dirección específica dentro de una API donde se puede acceder a un recurso o ejecutar una operación determinada mediante una solicitud HTTP.

Framework: conjunto de herramientas, bibliotecas y buenas prácticas que facilitan el desarrollo de aplicaciones al proporcionar estructuras predefinidas para organizar el código.

HTTP (Hypertext Transfer Protocol): protocolo de comunicación utilizado en la web que permite el intercambio de información entre clientes (como navegadores) y servidores.

IDE (Entorno de Desarrollo Integrado): aplicación que reúne diferentes herramientas de programación, como editor de código, compilador, depurador y gestor de proyectos, para facilitar el desarrollo de software.

Idempotencia: propiedad de algunas operaciones en la que realizar la misma solicitud varias veces produce siempre el mismo resultado en el servidor.

JSON (JavaScript Object Notation): formato ligero de intercambio de datos basado en texto que permite representar información estructurada de forma sencilla y legible para humanos y máquinas.

Migración de base de datos: proceso mediante el cual se modifican, trasladan o actualizan estructuras de datos dentro de una base de datos, generalmente mediante scripts controlados que preservan la información existente.

ORM (Mapeo objeto-relacional): técnica de programación que permite interactuar con bases de datos relacionales utilizando objetos del lenguaje de programación en lugar de escribir consultas SQL directamente.

REST (Representational State Transfer): estilo de arquitectura para el desarrollo de servicios web que utiliza el protocolo HTTP para acceder y manipular recursos identificados mediante URLs.

Servidor: sistema informático que proporciona servicios, recursos o datos a otros sistemas llamados clientes a través de una red.

Solicitud HTTP: mensaje enviado por un cliente a un servidor para solicitar información o ejecutar una acción sobre un recurso disponible en una aplicación o API.

URL (Uniform Resource Locator): dirección única que identifica la ubicación de un recurso en Internet y permite acceder a él mediante un navegador o una aplicación.

Referencias bibliográficas

Avast. (2024). *Todo lo que necesita saber sobre las redes de área local*.

<https://www.avast.com/es-es/c-what-is-lan>

Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3rd ed.). Addison-Wesley.

BusinessCom. (s. f.). *Acceso a Internet por Satélite*.

<https://www.bcsatellite.net/es/acceso-a-internet-por-satelite/>

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).

Fernández, Y. (2026). *Cable coaxial: qué es, para qué sirve, tipos y cuál elegir*.

Xataka Basics. <https://www.xataka.com/basics/cable-coaxial-que-sirve-tipos-cual-elegir>

Fowler, M. (2003). *Patterns of enterprise application architecture*. Addison-Wesley.

Gough, H. (2024). *Todo lo que necesita saber sobre las redes de área local*. Avast.

<https://www.avast.com/es-es/c-what-is-lan>

Noori, R. (2024). *¿De qué materiales están hechos los cables de fibra óptica?* The Network Installers. <https://thenetworkinstallers.com/es/blog/de-que-materiales-estan-hechos-los-cables-de-fibra-optica/>

Richardson, L., & Amundsen, M. (2013). *RESTful Web APIs*. O'Reilly Media.

Sebesta, R. W. (2016). *Concepts of programming languages* (11th ed.). Pearson.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Tilkov, S., & Vinoski, S. (2010). *Node.js: Using JavaScript to build high-performance network programs*. IEEE Internet Computing.

Ubigo. (2024). *¿Qué son las antenas celulares?*
<https://www.ubigo.net/tecnologia/antenas-celulares/>

Webformix. (2021). *Fixed Wireless Internet vs DSL Internet: Pros & Cons*.
<https://www.webformix.com/fixed-wireless-internet-vs-dsl-internet-pros-cons/>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Veimar Celis Meléndez	Experto temático	Centro de Comercio y Servicios - Regional Tolima
Viviana Esperanza Herrera Quiñonez	Evaluada instrucción	Centro de Comercio y Servicios - Regional Tolima
Oscar Ivan Uribe Ortiz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Francisco José Vásquez Suárez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Sebastian Trujillo Afanador	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Evalua de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluada de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima